

Recurso 1

Título del recurso relacionado: Del monolítico al cliente-servidor distribuido: guía conceptual paso a paso

Descripción del recurso: Hoja de ruta narrada para entender cómo se pasa de una aplicación monolítica a una arquitectura cliente-servidor moderna con servicios especializados. Incluye hitos, decisiones y riesgos típicos.

Objetivo de la guía

Que el/la estudiante visualice la evolución arquitectónica y entienda qué problemas resuelve cada transición, sin implementar código.

Paso a paso

Paso 1. Punto de partida: monolito.

Se parte de una sola base de código con UI, lógica y datos acoplados. *Ventaja:* simplicidad inicial. *Dolor:* despliegues lentos, una falla afecta todo.

Paso 2. Separación lógica por capas.

Se introduce la división conceptual (presentación, negocio, datos). Aunque sigue en un único binario, aparece la idea de responsabilidades. *Aprendizaje:* la modularidad reduce el impacto del cambio.

Paso 3. Primera escisión: cliente vs servidor.

La UI pasa a ejecutarse en el cliente (navegador/escritorio) y la lógica a un servidor accesible por red (HTTP/HTTPS). *Beneficio:* centralizar reglas y actualizar sin reinstalar clientes.

Paso 4. Contratos estables (APIs).

Se formalizan endpoints REST/GraphQL como “contratos”. *Claves:* versionado de API, documentación; *riesgo:* romper compatibilidad.

Paso 5. Escalado horizontal con balanceo.

Se replica el servidor detrás de un *load balancer* (L7). *Idea crítica:* estado de sesión fuera del proceso (cookies firmadas/JWT, caché central).

Paso 6. Datos distribuidos y caché.

Se introducen *read replicas*, *sharding* y cachés (ej. Redis) para reducir latencia. *Trade-off:* coherencia vs rendimiento.

Paso 7. Observabilidad y resiliencia.

Se agregan *logs estructurados*, métricas (p99 de latencia), *tracing* distribuido y patrones de resiliencia (circuit breaker, *timeouts*, reintentos exponenciales).

Paso 8. Endurecimiento de seguridad.

TLS en tránsito, *secret management*, políticas CORS, autenticación (OAuth2/OIDC), autorización por *scopes/roles* y WAF para amenazas OWASP.

Resultado esperado (conceptual): Comprender *qué problema resuelve cada salto* y *qué nuevo riesgo introduce*, para decidir con criterio en los retos.

Sugerencia visual

Infografía “Evolución arquitectónica” (creación propia): línea de tiempo con iconografía de monolito → 3 capas → cliente-servidor → balanceo → caché/replicas → observabilidad/seguridad.

Recurso 2

Título del recurso relacionado: Del request al response: recorrido de una solicitud en una arquitectura cliente-servidor actual

Descripción del recurso: Mapa narrado de extremo a extremo: qué sucede desde que el cliente hace clic hasta que ve la respuesta. Incluye responsabilidades por capa y señales de calidad.

Objetivo de la guía

Hacer visible el “camino” de una petición para relacionar los conceptos de la Arquitectura cliente-servidor moderna con decisiones de diseño y pruebas.

Paso a paso

Paso 1. DNS y CDN.

El dominio se resuelve a una IP; una CDN puede servir assets estáticos cerca del usuario.

Indicador: cache hit ratio alto.

Paso 2. Handshake TLS (HTTPS).

Negociación de cifrado y certificados válidos. *Indicador:* ningún contenido mixto; *riesgo:* certificados expirados.

Paso 3. Balanceador de carga (L7).

Distribuye tráfico por *health checks* y políticas (round-robin/least-connections). *Indicador:* error rate homogéneo entre instancias.

Paso 4. Reverse proxy y caché de aplicación.

Normaliza cabeceras, comprime, aplica reglas; puede cachear respuestas idempotentes.

Beneficio: latencias más bajas.

Paso 5. Capa de aplicación.

Controladores validan entrada, coordinan dominio y orquestan servicios. *Buenas prácticas:* *timeouts*, *circuit breakers*, idempotencia en POST críticos.

Paso 6. Persistencia y consultas.

Estrategias de lectura/escritura (replicas, colas asíncronas). *Riesgo:* *N+1 queries*; *mitigación:* *joins* adecuados o agregaciones.

Paso 7. Seguridad de sesión y CORS.

Tokens (JWT) firmados/rotados; CORS restrictivo. *Indicador:* cero *preflight* fallidos y revocación de tokens efectiva.

Paso 8. Observabilidad.

Trazas con *correlation id*, métricas (RPS, p95/p99, saturación), alertas SLO. *Meta*: detectar regresiones antes que el usuario.

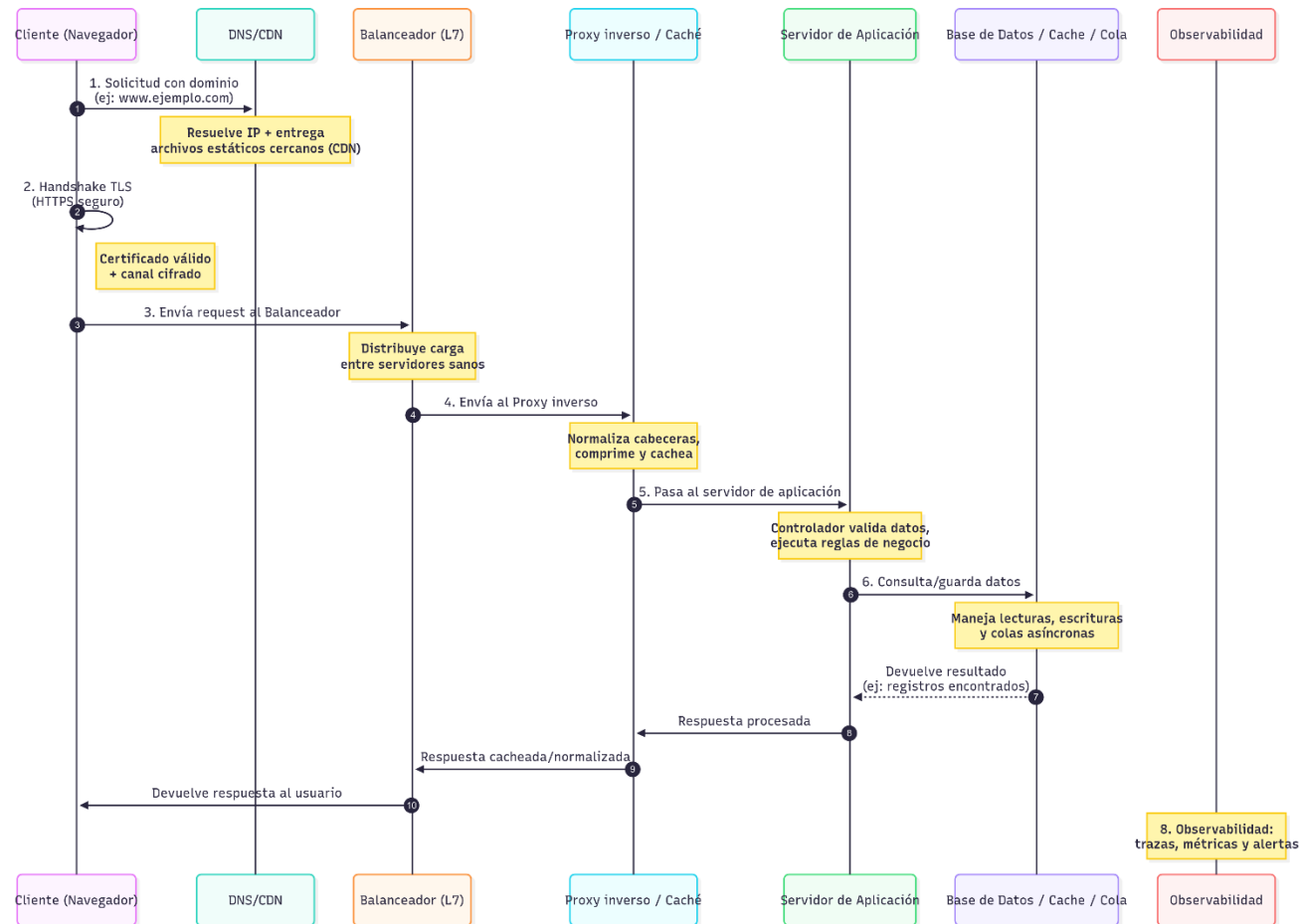
Tabla de referencia rápida

Tabla A. Capas, objetivos, fallos comunes y métricas clave

Capa	Objetivo principal	Fallos comunes	Métrica clave
DNS/CDN	Cercanía y cache de estáticos	TTL mal ajustado, <i>miss</i> constante	Cache hit ratio
TLS/HTTPS	Confidencialidad/Integridad	Certificado inválido/mixto	% tráfico en HTTPS
Balanceador (L7)	Distribución y <i>health checks</i>	<i>Hot spotting</i> , <i>sticky</i> excesivo	Error rate por instancia
Reverse proxy / Caché	Normalizar y acelerar	Reglas contradictorias	Latencia p95
App (controlador/dominio)	Ejecutar reglas de negocio	<i>Timeouts</i> ausentes, N+1 queries	Tasa de errores 5xx
Datos (DB/Cache/Queue)	Persistencia y performance	Bloqueos, contención, <i>thundering</i>	QPS, locks, cache hit
Seguridad (Auth/CORS)	Identidad y acceso	Tokens sin rotación, CORS laxo	Fallos de auth/autz
Observabilidad	Visibilidad y alerta temprana	Sin <i>correlation id</i>	MTTR/MTTD

Sugerencia visual

Diagrama de secuencia (creación propia): Cliente → CDN → LB → Proxy → App → DB/Cache → App → Cliente, con anotaciones de cabeceras y tiempos.



Recurso 3

Título del recurso relacionado: HTML5 semántico + WCAG 2.2 AA: checklist comentada y ejemplos canónicos

Descripción del recurso: Lista guiada para evaluar semántica y accesibilidad sin herramientas externas, con ejemplos típicos (navegación, tarjetas, formularios) y notas ARIA.

Objetivo de la guía

Ofrecer una ruta clara para reconocer buen marcado semántico y criterios AA de accesibilidad, con casos reales de interfaz.

Paso a paso (checklist comentada)

Paso 1. Estructura global correcta.

Un solo `<main>` por página; `<header>`/`<footer>` por sitio o sección.

Paso 2. Navegación identificable.

Usar `<nav>`; si hay varias, diferenciarlas con `aria-label` ("Navegación principal", "Breadcrumbs").

Paso 3. Jerarquía de encabezados.

Un `<h1>` por página; subsecciones con `<h2>`...`<h6>` en orden lógico (no para estilos).

Paso 4. Landmarks y roles.

Preferir elementos semánticos; si no hay, añadir role apropiado (por ejemplo, `role="complementary"` para un `<aside>` complejo).

Paso 5. Imágenes con propósito.

`alt` descriptivo para imágenes informativas; `alt=""` para decoración pura.

Paso 6. Formularios accesibles.

Cada `<input>` con `<label for>` o `aria-label`; *hint* con `aria-describedby`; mensajes de error vinculados al control.

Paso 7. Estado y foco visibles.

No eliminar *focus outline* sin reemplazo perceptible; *tab order* coherente.

Paso 8. Contraste de color.

Texto normal $\geq 4.5:1$; texto grande ($\geq 18\text{px}$ o 14px negrita) $\geq 3:1$. Evitar texto sobre imágenes sin *overlay*.

Paso 9. Componentes interactivos.

No usar <div> clicables: preferir <button>/<a> según corresponda. Añadir aria-pressed, aria-expanded si aplica.

Paso 10. Contenido dinámico.

Para actualizaciones, usar aria-live="polite" (no bloquear lectura) o assertive si es crítico. Evitar *announcements* redundantes.

Paso 11. Tablas de datos.

<th scope="col/row"> para cabeceras; caption para describir propósito; evitar tablas para layout.

Paso 12. Enlaces significativos.

El texto del enlace debe describir destino (“Ver detalles de matrícula”), no “clic aquí”.

Paso 13. Accesibilidad por teclado.

Todo alcanzable con Tab/Enter/Space. *Trap focus* manejado en modales con aria-modal="true".

Paso 14. Semántica en componentes de tarjeta.

Usar <article> para una tarjeta independiente; título en <h3> dentro de la tarjeta; acciones con <button>.

Paso 15. Validación básica.

Comprobar con atajos (Lighthouse/Axe) y luego inspección manual: *skip links*, foco lógico y lectura con lector de pantalla.

Tabla de mapeo útil

Tabla B. Patrón de UI → Elementos semánticos → ARIA sugerido → Falla común

Patrón UI	Elementos semánticos	ARIA sugerido	Falla común
Menú principal	<nav> + lista	aria-label="Principal"	<div> suelto con <i>onclick</i>
Tarjeta de noticia	<article> + <h3>	—	Encabezados saltados (h1→h4)

Modal	<dialog>/ contenedor	aria-modal, aria- labelledby	Foco sin retorno al cierre
Accordion	<button> + región	aria-expanded, aria- controls	Solo estilos; sin semántica ni foco
Formulario	<form> + <label>	aria-describedby	Inputs sin label ni mensajes de error

Resultado esperado (conceptual): Reconocer, sin “hacer”, cómo luce un documento correcto y dónde suelen aparecer los errores que rompen la accesibilidad.

Ejemplo canónico (ilustrativo, no actividad)

Fragmento de formulario correctamente asociado (con *hints* y error anclado).

```
<form aria-labelledby="tituloseccion">

  <h2 id="tituloseccion">Registro</h2>

  <label for="correo">Correo institucional</label>

  <input id="correo" type="email" aria-describedby="ayuda-correo error-correo">

  <p id="ayuda-correo">Usa tu cuenta @ejemplo.edu.ec</p>

  <p id="error-correo" role="alert">Formato inválido</p>

  <button type="submit">Enviar</button>

</form>
```